Mise en place de la plateforme Seahawks Monitoring

1. Objectif de la mise en œuvre

L'objectif de cette phase était de déployer une plateforme de supervision distribuée composée de deux machines virtuelles Linux :

- **Harvester** : sonde locale chargée de collecter les informations réseau du site
- **Nester** : plateforme centrale chargée de recevoir, stocker et afficher les rapports

Cette architecture permet de simuler un fonctionnement multisite dans lequel une sonde locale supervise son environnement et transmet les données vers un centre de consultation.

2. Préparation de l'environnement

La première étape a consisté à préparer les deux machines virtuelles sous Ubuntu Server, l'une pour la sonde **Harvester**, l'autre pour le serveur central **Nester**.

Après l'installation du système, nous avons procédé à la mise en place des outils de base nécessaires au projet. Sur les deux machines, les paquets suivants ont été installés :

- python3
- python3-pip
- git
- curl
- nano
- net-tools

Cette étape permet de disposer :

- d'un environnement Python pour exécuter les scripts,
- d'outils de configuration et d'administration,
- de commandes réseau utiles pour les tests et le diagnostic.

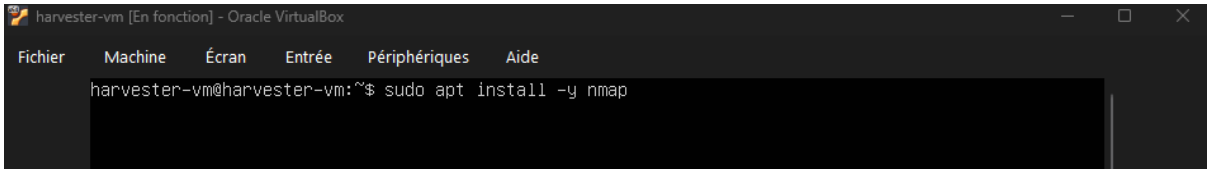
3. Installation des dépendances spécifiques

Une fois l'environnement de base prêt, nous avons installé les dépendances propres à chaque rôle.

3.1. Sur Harvester

Sur la machine **Harvester**, nous avons installé :

- nmap
- la bibliothèque Python python-nmap



L'outil **Nmap** constitue le moteur de scan réseau utilisé par la sonde.
La bibliothèque **python-nmap** permet au script Python d'exploiter Nmap directement depuis le code afin de découvrir les hôtes actifs sur le réseau.

3.2. Sur Nester

Sur la machine **Nester**, nous avons installé la bibliothèque :

- Flask

Cette bibliothèque sert à développer la plateforme web centrale.
Elle permet de mettre en place :

- une interface de consultation,
- un point d'entrée HTTP pour recevoir les rapports,
- un affichage centralisé des données envoyées par les sondes.

4. Configuration réseau des machines

```
nester-vm@nester-vm:~$ sudo netplan apply

** (generate:938): WARNING **: 09:16:09.822: Permissions for /etc/netplan/00-installer-config.yaml a
re too open. Netplan configuration should NOT be accessible by others.
WARNING:root:Cannot call Open vSwitch: ovsdb-server.service is not running.

** (process:936): WARNING **: 09:16:11.069: Permissions for /etc/netplan/00-installer-config.yaml ar
e too open. Netplan configuration should NOT be accessible by others.

** (process:936): WARNING **: 09:16:11.642: Permissions for /etc/netplan/00-installer-config.yaml ar
e too open. Netplan configuration should NOT be accessible by others.

** (process:936): WARNING **: 09:16:11.642: Permissions for /etc/netplan/00-installer-config.yaml ar
e too open. Netplan configuration should NOT be accessible by others.
nester-vm@nester-vm:~$ sudo chmod 600 /etc/netplan/00-installer-config.yaml
nester-vm@nester-vm:~$ sudo netplan apply
WARNING:root:Cannot call Open vSwitch: ovsdb-server.service is not running.
nester-vm@nester-vm:~$ _
```

Afin de permettre la communication entre les deux machines virtuelles, nous avons configuré le réseau avec deux interfaces :

- une interface **NAT** pour l'accès Internet et l'installation des paquets,
- une interface **réseau interne** pour la communication privée entre Harvester et Nester.

La configuration réseau a été appliquée avec **Netplan**.
Lors de cette étape, un avertissement lié aux permissions du fichier de configuration est apparu. Nous l'avons corrigé en adaptant les droits du fichier /etc/netplan/00-installer-config.yaml, puis en réappliquant la configuration réseau.

Après application, nous avons vérifié les interfaces avec la commande :

ip a

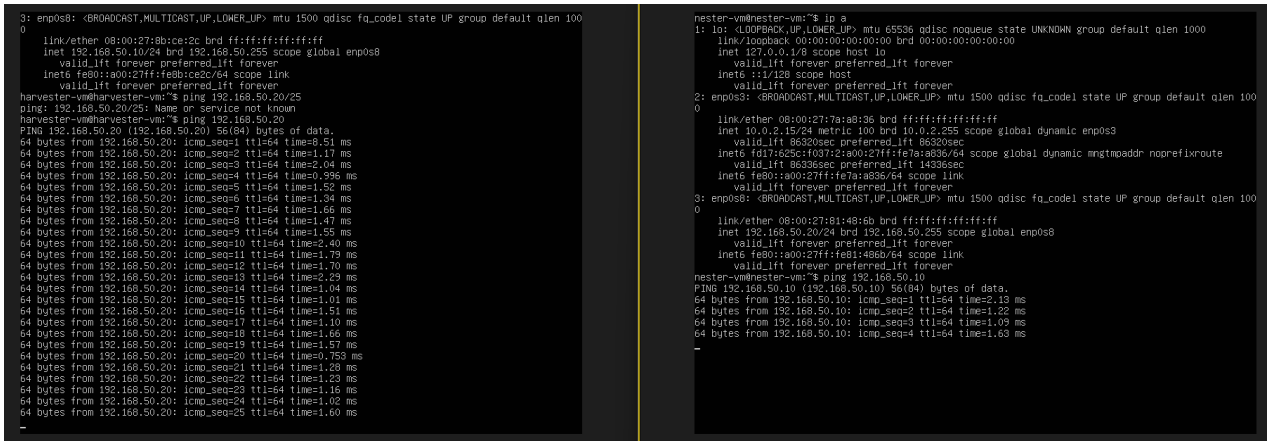
Cette vérification a permis de confirmer la présence des deux adresses IP internes :

- **Harvester** : 192.168.50.10
- **Nester** : 192.168.50.20

Cette étape était essentielle pour garantir la bonne communication entre la sonde et la plateforme centrale.

5. Vérification de la connectivité

Une fois la configuration réseau appliquée, nous avons testé la communication entre les deux machines.



Les captures montrent que chaque machine dispose bien :

- d’une interface NAT (10.0.2.15)
- d’une interface interne (192.168.50.x)

Cela confirme que :

- les deux VM ont accès à Internet si nécessaire,
- elles peuvent également communiquer entre elles sur le réseau interne du projet.

Cette double interface permet de reproduire un cas réaliste de supervision réseau avec une séparation entre réseau de gestion et réseau de service.

6. Création de la structure du projet Harvester

Sur la machine **Harvester**, nous avons ensuite créé l’arborescence du projet :

- un dossier principal seahawks-harvester
- un dossier reports pour stocker les rapports JSON générés
- un dossier logs pour stocker les journaux d’exécution

Cette organisation permet de séparer clairement :

- le code de la sonde,
- les données produites,
- les fichiers de log utilisés pour le diagnostic.

Ce choix améliore la lisibilité, la maintenabilité et l'exploitabilité de la solution.

```
harvester-vm@harvester-vm:~$ mkdir seahawks-harvester
harvester-vm@harvester-vm:~$ cd seahawks-harvester
harvester-vm@harvester-vm:~/seahawks-harvester$ mkdir reports logs
harvester-vm@harvester-vm:~/seahawks-harvester$ _
```

7. Développement du composant Harvester

Le script scanner.py a ensuite été mis en place sur la machine Harvester.

MSPR1-NFL-IT/harvester at main · AnasElmaliki69/MSPR1-NFL-IT

Ce script assure plusieurs fonctions :

- scan du sous-réseau local,
- détection des hôtes actifs,
- mesure de la latence vers la plateforme centrale,
- génération d'un rapport structuré en JSON,
- journalisation des exécutions,
- envoi sécurisé du rapport vers Nester.

Une fois exécuté, le script produit un fichier JSON contenant notamment :

- l'identifiant de la machine (host_id)
- la date et l'heure du scan
- le nombre d'équipements détectés
- la liste des hôtes actifs
- la latence moyenne vers le serveur central
- la version du composant

```
harvester-vm@harvester-vm:~/seahawks-harvester$ ls reports
report_20260316_101853.json
harvester-vm@harvester-vm:~/seahawks-harvester$ cat reports/report_20260316_101853.json
{
    "host_id": "harvester-vm",
    "scan_date": "2026-03-16 10:18:53.442382",
    "devices_detected": 2,
    "hosts": [
        "192.168.50.10",
        "192.168.50.20"
    ],
    "version": "1.0"
```

La capture montre qu'un premier rapport a bien été généré localement avec les hôtes :

- 192.168.50.10
- 192.168.50.20

Cela valide la collecte réseau locale ainsi que la persistance des données sur la sonde.

8. Journalisation et mesure de latence

Le composant Harvester a ensuite été enrichi par un mécanisme de journalisation.

Les fichiers de logs permettent d'enregistrer :

- le nombre d'équipements détectés,
- la latence mesurée,
- les erreurs éventuelles,
- les échecs de transmission.

Lors des tests, un premier scan a montré une latence **injoignable** lorsque la machine Nester était éteinte.
Après redémarrage de Nester, un second scan a affiché une latence correcte de l'ordre de quelques millisecondes.

```
cat: reports/newfile.json: No such file or directory
harvester-vm@harvester-vm:~/seahawks-harvester$ cat logs/harvester.log
2026-03-16 12:25:59,730 INFO Scan completed. Devices: 1 Latency: unreachable
2026-03-16 12:32:14,826 INFO Scan completed. Devices: 2 Latency: 2.866
harvester-vm@harvester-vm:~/seahawks-harvester$
```

Ce comportement valide la capacité du système à :

- détecter une indisponibilité,
- journaliser l'événement,
- se mettre à jour automatiquement lorsque le service redevient accessible.

9. Mise en place de la plateforme centrale Nester

Sur la machine **Nester**, nous avons développé un serveur Flask dans le dossier seahawks-nester.

MSPR1-NFL-IT/nester at main · AnasElmaliki69/MSPR1-NFL-IT

Cette application remplit plusieurs fonctions :

- recevoir les rapports JSON via une route /upload,
- stocker les rapports dans un dossier local,
- afficher un tableau de bord web,
- consulter le détail d'un rapport,
- indiquer l'état d'une sonde (connectée ou déconnectée).

La plateforme a été lancée avec :

python3 app.py

```
nester-vm@nester-vm:~/seahawks-nester$ python3 app.py
 * Serving Flask app 'app'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WS
GI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://10.0.2.15:5000
Press CTRL+C to quit
```

Les captures montrent que le serveur Flask écoute correctement sur le port 5000 et qu'il reçoit les requêtes HTTP en provenance de la machine Harvester.

10. Validation de la communication Harvester → Nester

Une fois les deux composants en place, nous avons testé le flux complet :

1. exécution du script Harvester,
2. génération du rapport JSON local,
3. envoi HTTP vers Nester,
4. réception côté plateforme centrale.

Les journaux du serveur Flask montrent bien une requête :

POST /upload HTTP/1.1 200

Ce code **200** confirme que :

- la requête a bien été reçue,
- le rapport a été accepté,
- la chaîne de communication entre les deux composants est fonctionnelle.

```
harvester-vm@harvester-vm:~/seahawks-harvester$ python3 scanner.py
Scan finished
{'host_id': 'harvester-vm', 'scan_date': '2026-03-16 13:42:05.939518', 'devices_detected': 2, 'hosts
': ['192.168.50.10', '192.168.50.20'], 'average_latency_ms': '2.464', 'version': '1.1'}
harvester-vm@harvester-vm:~/seahawks-harvester$ _
```

```
nester-vm@nester-vm:~/seahawks-nester$ python3 app.py
 * Serving Flask app 'app'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WS
GI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://10.0.2.15:5000
Press CTRL+C to quit
192.168.50.10 - - [16/Mar/2026 13:42:06] "POST /upload HTTP/1.1" 200 -
```

Cette étape valide le cœur fonctionnel de la plateforme.

11. Sécurisation de la transmission

Afin d'éviter qu'un poste non autorisé puisse envoyer de faux rapports au serveur central, nous avons ajouté une **clé API** dans les échanges entre Harvester et Nester.

Le principe est le suivant :

- Harvester envoie la requête HTTP avec un en-tête X-API-KEY
- Nester vérifie cette clé avant d'accepter le rapport
- si la clé est incorrecte, le serveur renvoie une erreur **401 Unauthorized**

Les tests ont montré :

- un premier envoi accepté avec le code **200**
- un second essai rejeté avec le code **401**

```
nester-vm@nester-vm:~/seahawks-nester$ python3 app.py &
[1] 1511
nester-vm@nester-vm:~/seahawks-nester$  * Serving Flask app 'app'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WS
GI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://10.0.2.15:5000
Press CTRL+C to quit
192.168.50.10 - - [18/Mar/2026 21:26:35] "POST /upload HTTP/1.1" 200 -
192.168.50.10 - - [18/Mar/2026 21:29:21] "POST /upload HTTP/1.1" 401 -
```

Cette mesure constitue un premier niveau de sécurisation des flux applicatifs.

12. Mise en place d'une supervision centralisée

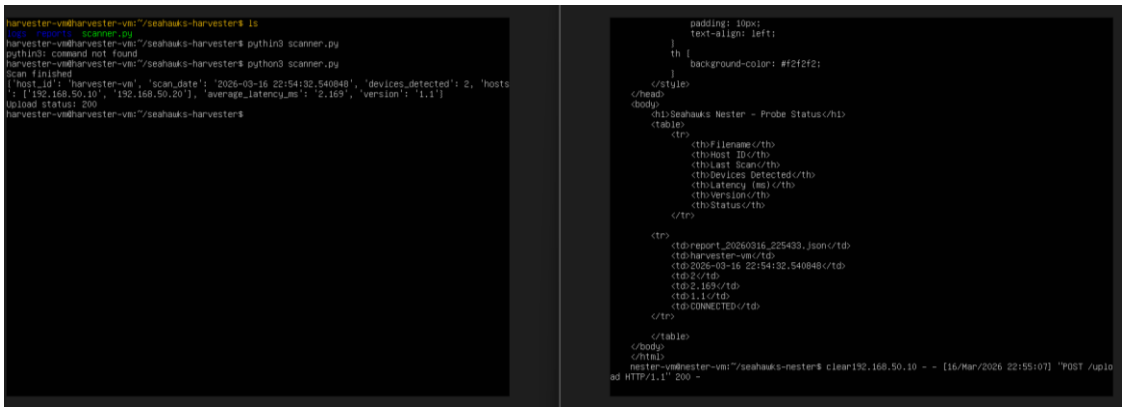
La page principale de la plateforme Nester a été conçue pour afficher les informations principales de chaque sonde :

- nom du fichier de rapport,

- identifiant de la sonde,
- date du dernier scan,
- nombre d’équipements détectés,
- latence,
- version,
- état de connexion.

Le système détermine automatiquement si une sonde est :

- **CONNECTED** si le dernier rapport est récent,
- **DISCONNECTED** si aucun rapport récent n’a été reçu.



Cette logique permet de simuler une supervision temps réel orientée support.

13. Consultation détaillée des rapports

En complément du tableau de bord principal, une page de détail a été ajoutée.

Cette page permet de cliquer sur un rapport et d’afficher :

- son contenu JSON complet,
- les informations de diagnostic,
- les mesures collectées lors du dernier scan.

Cette fonctionnalité améliore l’exploitabilité de la plateforme et facilite l’analyse en cas d’incident.

Seahawks Nester - Probe Status

Filename	Host ID	Last Scan	Devices Detected	Latency (ms)	Version	Status	Alert
report_20260319_202611.json	harvester-vm	2026-03-19 20:26:10.834835	2	0.919	1.2	CONNECTED	OK

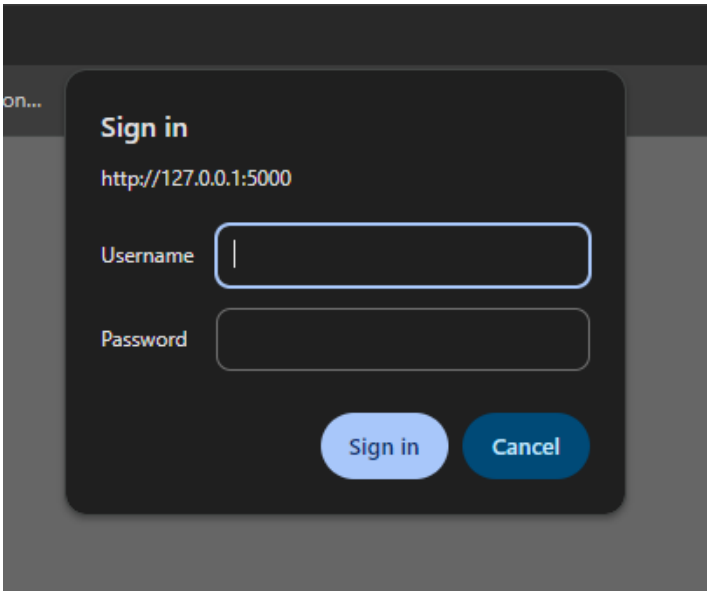
14. Authentification à l'interface web

Pour limiter l'accès aux informations de supervision, un mécanisme d'authentification a été ajouté sur l'interface web de Nester.

L'objectif de cette mesure est de :

- restreindre l'accès aux données sensibles,
- empêcher une consultation non autorisée,
- renforcer la dimension cybersécurité du projet.

Cela montre que la plateforme ne se limite pas à la collecte, mais intègre également des mécanismes de protection d'accès.



15. Automatisation du composant Harvester

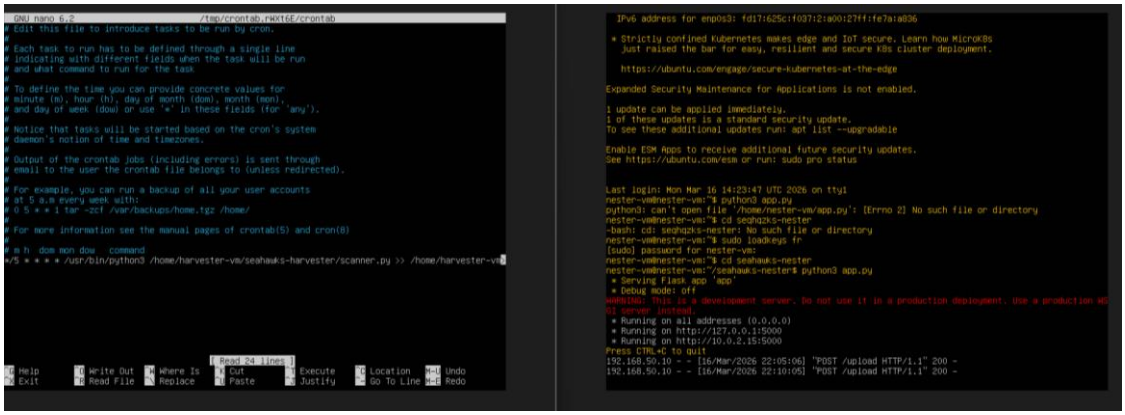
Une fois le fonctionnement manuel validé, nous avons cherché à rendre la sonde autonome.

Deux approches ont été testées :

15.1. Exécution planifiée avec cron

Dans un premier temps, le script a été exécuté automatiquement via une tâche cron.
Cette méthode permettait de lancer un scan périodique sans intervention humaine.

Des ajustements ont été nécessaires sur les chemins absolus, car le contexte d'exécution de cron diffère d'une exécution manuelle.



15.2. Exécution via un service systemd

Dans un second temps, nous avons mis en place un service systemd pour exécuter Harvester comme un véritable agent système.

Le fichier harvester.service définit notamment :

- le nom du service,
- le répertoire de travail,
- le script à exécuter,
- la politique de redémarrage automatique.

Cette approche est plus professionnelle et plus proche d'un déploiement réel, car elle permet :

- un démarrage automatique,
- une meilleure supervision du service,
- un redémarrage en cas d'échec.

```
harvester-vm@harvester-vm:/etc/systemd/system$ sudo cat harvester.service
[Unit]
Description=Seahawks Harvester Monitoring Agent
After=network.target

[Service]
User=harvester-vm
WorkingDirectory=/home/harvester-vm/seahawks-harvester
ExecStart=/usr/bin/python3 /home/harvester-vm/seahawks-harvester/scanner.py
Restart=always
RestartSec=60

[Install]
WantedBy=multi-user.targetharvester-vm@harvester-vm:/etc/systemd/system$ _
```

16. Activation du service Harvester

Après création du fichier de service, nous avons exécuté les commandes suivantes :

- systemctl daemon-reload
- systemctl enable harvester
- systemctl start harvester
- systemctl status harvester

Les captures montrent que le service a bien été chargé et activé.
Le statut indique que le script est lancé, puis redémarré automatiquement selon la politique définie.

```
harvester-vm@harvester-vm:~$ sudo systemctl daemon-reload
harvester-vm@harvester-vm:~$ sudo systemctl enable harvester
Created symlink /etc/systemd/system/multi-user.target.wants/harvester.service → /etc/systemd/system/
harvester.service.
harvester-vm@harvester-vm:~$ sudo systemctl start harvester
harvester-vm@harvester-vm:~$ sudo systemctl status harvester
● harvester.service - Seahawks Harvester Monitoring Agent
     Loaded: loaded (/etc/systemd/system/harvester.service; enabled; vendor preset: enabled)
     Active: activating (auto-restart) since Wed 2026-03-18 22:35:45 UTC; 19s ago
    Process: 1600 ExecStart=/usr/bin/python3 /home/harvester-vm/seahawks-harvester/scanner.py (code>
   Main PID: 1600 (code=exited, status=0/SUCCESS)
        CPU: 513ms
lines 1-6/6 (END)_
```

Cette étape confirme que la sonde peut fonctionner comme un **agent persistant**, sans lancement manuel.

17. Résultat final

À l’issue de cette mise en place, la plateforme obtenue permet :

- de collecter automatiquement des informations réseau locales,
- de mesurer la connectivité vers le centre,
- de générer des rapports JSON structurés,
- de centraliser les rapports sur une plateforme web,
- d'afficher l'état des sondes,
- de sécuriser l'envoi des données,
- d'automatiser le fonctionnement de la sonde.

Le système obtenu constitue un **prototype fonctionnel de supervision multisite**, cohérent avec les objectifs du projet Seahawks Monitoring.

18. Conclusion

La mise en œuvre de cette plateforme a permis de concrétiser les principaux objectifs techniques du cahier des charges :

- standardisation de la collecte,
- centralisation de la supervision,
- exploitabilité pour le support,
- intégration de mécanismes de sécurité,
- continuité de service grâce à l'automatisation.